

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 9

Date: 01-10-2026

Implementation of Mini Compiler using Lex and Yacc

Aim:

To implement a mini compiler with Lex and Yacc that takes a C++ source program and passes it through lexical analysis, syntax analysis, intermediate code generation, code optimization and code generation, producing 8086-style assembly code.

Tools Used:

- Lex (Flex) - recognizes keywords, identifiers, constants, operators and special characters as tokens, and reports lexical errors.
- Yacc (Bison) - defines the grammar, checks the syntax and declarations, and generates three address code through syntax directed translation rules.
- C Programming Language - used to write the symbol table, the code optimizer and the code generator.
- C Compiler (GCC/CC) - compiles the C code generated by Lex and Yacc together with the other C files.
- Linux Terminal (Ubuntu on WSL) - used to run the Lex, Yacc and gcc commands.
- C++ Input File (input.cpp) - the source program given to the compiler as input.

Code:

scanner.l

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include "y.tab.h"

/* Phase 1 : every token is listed together with its class */
static void show(const char *kind)
{
    printf("%s - %s\n", yytext, kind);
}

static void lexError(const char *msg)
{
    printf("Lexical error at line %d: %s\n", yylineno, msg);
    exit(1);
}
}%

%option noyywrap nounput noinput yylineno
%x COMMENT

DIGIT    [0-9]
LETTER   [a-zA-Z_]

%%

"//".*      { /* single line comment */ }
```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"/*"          { BEGIN(COMMENT); }
<COMMENT>"/" { BEGIN(INITIAL); }
<COMMENT>\n   { }
<COMMENT>.    { }
<COMMENT><<EOF>> { lexError("unterminated comment"); }

^[ \t]*"#"[^\\n]* { printf("%s - preprocessor directive\n",
                          yytext + strspn(yytext, " \t")); }

"using"       { show("keyword"); return USING; }
"namespace"   { show("keyword"); return NAMESPACE; }
"int"         { show("keyword"); return INT; }
"void"        { show("keyword"); return VOID; }
"return"      { show("keyword"); return RETURN; }
"if"          { show("keyword"); return IF; }
"else"        { show("keyword"); return ELSE; }
"while"       { show("keyword"); return WHILE; }
"main"        { show("identifier"); return MAIN; }
"std::"?"cout" { show("output stream (function call)"); return COUT; }
"std::"?"endl" { show("stream manipulator"); return ENDL; }
"std::"?"pow"  { show("function call"); return POWFN; }

{LETTER}{(LETTER)|(DIGIT)}* { show("identifier");
                              yylval.str = strdup(yytext); return ID; }

{DIGIT}+ { if (strlen(yytext) > 10 || atoll(yytext) > INT_MAX)
            lexError("integer constant too large");
            show("integer constant");
            yylval.str = strdup(yytext); return NUM; }

\"([^\\"\\n]|\\.)*\" { show("string constant");
                    yylval.str = strdup(yytext); return STR; }
\"([^\\"\\n]|\\.)* { lexError("unterminated string"); }

"<<" { show("stream insertion operator"); return SHL; }
"<="|">="|"=="|"!="|"<"|>" { show("relational operator");
                              yylval.str = strdup(yytext); return RELOP; }
"=" { show("assignment operator"); return '='; }
["+/*%] { show("arithmetic operator"); return yytext[0]; }
["(){}\\[\\];,] { show("special character"); return yytext[0]; }

[ \\t\\r\\n]+ { /* skip white space */ }
. { char m[40];
  sprintf(m, "unknown character '%s'", yytext);
  lexError(m); }

%%

```

parser.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "compiler.h"

int yylex(void);
void yyerror(const char *s);
extern int  yylineno;
extern char *yytext;
extern FILE *yyin;

static int phase2Started = 0;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

static void startPhase2(void)
{
    if (!phase2Started) printf("\n----- PHASE 2: SYNTAX ANALYSIS ----- \n");
    phase2Started = 1;
}

/* ----- symbol table */

#define MAXSYM 200

typedef struct {
    char name[NAMELEN];
    char type[12];
    int bytes, addr;
    char value[NAMELEN]; /* initial value if it is a constant, else "-" */
    int depth;           /* block in which the variable is declared */
    int hidden;          /* the block has ended: no longer visible */
    int assigned;        /* a value has been given to the variable */
} Sym;

static Sym sym[MAXSYM];
static int nsym = 0, nextAddr = 1000, depth = 0;

static int findSym(const char *name)
{
    int k;
    for (k = nsym - 1; k >= 0; k--)
        if (!sym[k].hidden && !strcmp(sym[k].name, name)) return k;
    return -1;
}

static void semError(const char *msg, const char *name)
{
    startPhase2();
    printf("Semantic error at line %d: %s '%s'\n", yylineno, msg, name);
    exit(1);
}

static void addSym(const char *name, const char *value, int assigned)
{
    if (findSym(name) >= 0) semError("variable already declared:", name);
    if (isTemp(name)) semError("name is reserved for temporaries:", name);
    if (nsym >= MAXSYM) semError("too many variables:", name);
    strcpy(sym[nsym].name, name);
    strcpy(sym[nsym].type, "int");
    sym[nsym].bytes = 4;
    sym[nsym].addr = nextAddr;
    nextAddr += 4;
    strcpy(sym[nsym].value, value);
    sym[nsym].depth = depth;
    sym[nsym].hidden = 0;
    sym[nsym].assigned = assigned;
    nsym++;
}

static void endBlock(void)
{
    int k;
    for (k = 0; k < nsym; k++)
        if (sym[k].depth == depth) sym[k].hidden = 1;
    depth--;
}

static void checkDeclared(const char *name)
{
    if (findSym(name) < 0) semError("undeclared variable", name);
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

static void checkUse(const char *name)
{
    int k = findSym(name);
    if (k < 0) semError("undeclared variable", name);
    if (!sym[k].assigned) semError("variable might not have been initialized:", name);
}

static void markAssigned(const char *name)
{
    sym[findSym(name)].assigned = 1;
}

/* ----- code generation helpers */

static char *binary(const char *op, const char *a, const char *b)
{
    char *t = newTemp();
    emit(op, a, b, t);
    return t;
}

static const char *negate(const char *rel) /* jump when the condition is false */
{
    if (!strcmp(rel, "<")) return ">=";
    if (!strcmp(rel, ">")) return "<=";
    if (!strcmp(rel, "<=")) return ">";
    if (!strcmp(rel, ">=")) return "<";
    if (!strcmp(rel, "==")) return "!=";
    return "=="
}
%}

%union { char *str; }

%token <str> ID NUM STR RELOP
%token USING NAMESPACE VOID MAIN INT RETURN IF ELSE WHILE COUT ENDL POWFN SHL
%type <str> expr cond

%left '+' '-'
%left '*' '/' '%'
%right UMINUS

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

%%

program
: usings INT MAIN '(' params ')' mainblock
;

/* using namespace std; (#include lines are removed by the scanner) */
usings
: usings USING NAMESPACE ID ';'
| /* empty */
;

params
: VOID
| /* empty */
;

/* body of main : statements, then an optional return expr; */
mainblock
: '{'
    stmts opt_return '}'
    { depth++; }
    { endBlock(); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

;

opt_return
: RETURN expr ';'
| /* empty */
;

block
: '{'                                { depth++; }
  stmts '}'                          { endBlock(); }
;

stmts
: stmts stmt
| /* empty */
;

stmt
: decl ';'
| assign
| ifstmt
| whilestmt
| print
| block
;

/* Declaration statement : int a, b = 5; */
decl
: INT declist
;

declist
: declitem
| declist ',' declitem
;

declitem
: ID                                { addSym($1, "-", 0); }
| ID '=' expr                      { addSym($1, isNum($3) ? $3 : "-", 1);
                                   emit("=", $3, "", $1); }
;

/* Assignment statement : a = b + 1; */
assign
: ID '=' expr ';'                  { checkDeclared($1);
                                   emit("=", $3, "", $1);
                                   markAssigned($1); }
;

/* Conditional statement : if (c) s | if (c) s else s */
ifstmt
: IF '(' cond ')' stmt %prec LOWER_THAN_ELSE
  { emit("label", "", "", $3); }
| IF '(' cond ')' stmt ELSE { char *end = newLabel();
  emit("goto", "", "", end);
  emit("label", "", "", $3);
  $<str>$ = end; }
  stmt                            { emit("label", "", "", $<str>7); }
;

/* Looping statement : while (c) s */
whilestmt
: WHILE                            { char *top = newLabel();
  emit("label", "", "", top);
  $<str>$ = top; }
  '(' cond ')' stmt                { emit("goto", "", "", $<str>2);
  emit("label", "", "", $4); }

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

;

/* Output statement : cout << "text" << a + b << endl; */
print
: COUT outlist '; '
;

outlist
: SHL outitem
| outlist SHL outitem
;

outitem
: STR { emit("print", $1, "", ""); }
| ENDL { emit("print", "\\n\\n\\n", "", ""); }
| expr { emit("print", $1, "", ""); }
;

/* the generated jump leaves the statement when the condition is FALSE */
cond
: expr RELOP expr { char *out = newLabel();
emit(negate($2), $1, $3, out);
$$ = out; }
;

expr
: expr '+' expr { $$ = binary("+", $1, $3); }
| expr '-' expr { $$ = binary("-", $1, $3); }
| expr '*' expr { $$ = binary("*", $1, $3); }
| expr '/' expr { $$ = binary("/", $1, $3); }
| expr '%' expr { $$ = binary("%", $1, $3); }
| POWFN '(' expr ',' expr ')' { $$ = binary("^", $3, $5); }
| '-' expr %prec UMINUS { char *t = newTemp();
emit("neg", $2, "", t);
$$ = t; }
| '(' expr ')' { $$ = $2; }
| NUM { $$ = $1; }
| ID { checkUse($1); $$ = $1; }
;

%%

void yyerror(const char *s)
{
startPhase2();
printf("Syntactically incorrect: %s near '%s' at line %d\\n", s, yytext, yylineno);
exit(1);
}

int main(int argc, char *argv[])
{
const char *path = (argc > 1) ? argv[1] : "input.cpp";
FILE *asmFile;
int k, before;

yyin = fopen(path, "r");
if (!yyin) {
printf("Cannot open %s\\n", path);
return 1;
}

printf("----- PHASE 1: LEXICAL ANALYSIS -----\\n");
yyparse();
startPhase2();
printf("Syntactically correct\\n");

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

printf("\n----- SYMBOL TABLE -----\n");
printf("%-12s %-8s %-8s %-10s %-10s\n", "Name", "Type", "Bytes", "Address", "Value");
for (k = 0; k < nsym; k++)
    printf("%-12s %-8s %-8d %-10d %-10s\n",
           sym[k].name, sym[k].type, sym[k].bytes, sym[k].addr, sym[k].value);

printf("\n----- PHASE 3: INTERMEDIATE CODE (TAC) -----\n");
printTAC();
before = liveCount();

printf("\n----- PHASE 4: OPTIMIZED CODE -----\n");
optimize();
printTAC();
printf("\n");
printOptimizationReport();
printf("Instructions: %d -> %d\n", before, liveCount());

printf("\n----- PHASE 5: TARGET CODE (8086 style) -----\n");
asmFile = fopen("output.asm", "w");
if (!asmFile) {
    printf("Cannot create output.asm\n");
    return 1;
}
generate(asmFile);
fclose(asmFile);
printf("\nAssembly code written to output.asm\n");
return 0;
}

```

compiler.h

```

/*
 * Mini compiler - shared declarations
 *
 * Intermediate code is kept as a table of quadruples (op, arg1, arg2, result):
 *
 * + - * / % ^      result = arg1 op arg2
 * neg             result = -arg1
 * =               result = arg1
 * < <= > >= == !=   if arg1 op arg2 goto result
 * goto            goto result
 * label           result :
 * print           print arg1    (cout << arg1)
 */
#ifndef COMPILER_H
#define COMPILER_H

#include <stdio.h>

#define MAXQ 1000 /* maximum number of quadruples */
#define NAMELEN 128 /* maximum length of a name or string */

typedef struct {
    char op[8];
    char a1[NAMELEN], a2[NAMELEN], res[NAMELEN];
    int del; /* 1 = removed by the optimizer */
} Quad;

extern Quad q[MAXQ];
extern int nq;

/* quad.c */
void emit(const char *op, const char *a1, const char *a2, const char *res);
char *newTemp(void);
char *newLabel(void);

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

int  isTemp(const char *s);          /* t1, t2, ...          */
int  isNum(const char *s);          /* integer constant    */
int  isStr(const char *s);          /* "string constant"   */
int  isVar(const char *s);          /* name of a source variable */
int  isBin(const char *op);         /* + - * / % ^        */
int  isRel(const char *op);         /* < <= > >= == !=      */
int  defines(const Quad *x);        /* quad assigns a value to x->res */
void printTAC(void);
int  liveCount(void);

/* optimizer.c */
void optimize(void);
void printOptimizationReport(void);

/* codegen.c */
void generate(FILE *asmFile);

#endif

```

quad.c

```

/* Quadruple table : storage, temporaries, labels and printing */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "compiler.h"

Quad q[MAXQ];
int  nq = 0;

static int tcount = 0, lcount = 0;

void emit(const char *op, const char *a1, const char *a2, const char *res)
{
    if (nq >= MAXQ) {
        printf("Error: program is too long\n");
        exit(1);
    }
    strcpy(q[nq].op, op);
    strcpy(q[nq].a1, a1);
    strcpy(q[nq].a2, a2);
    strcpy(q[nq].res, res);
    q[nq].del = 0;
    nq++;
}

char *newTemp(void)
{
    char buf[16];
    sprintf(buf, "t%d", ++tcount);
    return strdup(buf);
}

char *newLabel(void)
{
    char buf[16];
    sprintf(buf, "L%d", ++lcount);
    return strdup(buf);
}

int isTemp(const char *s)
{
    int k;
    if (s[0] != 't' || !s[1]) return 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    for (k = 1; s[k]; k++)
        if (!isdigit((unsigned char)s[k])) return 0;
    return 1;
}

int isNum(const char *s)
{
    if (*s == '-') s++;
    if (!*s) return 0;
    while (*s)
        if (!isdigit((unsigned char)*s++)) return 0;
    return 1;
}

int isStr(const char *s)
{
    return s[0] == '\'';
}

int isVar(const char *s)
{
    return (isalpha((unsigned char)s[0]) || s[0] == '_') && !isTemp(s);
}

int isBin(const char *op)
{
    return op[0] && !op[1] && strchr("+-*/%^", op[0]);
}

int isRel(const char *op)
{
    return !strcmp(op, "<") || !strcmp(op, ">") || !strcmp(op, "<=") ||
        !strcmp(op, ">=") || !strcmp(op, "==") || !strcmp(op, "!=");
}

int defines(const Quad *x)
{
    return isBin(x->op) || !strcmp(x->op, "neg") || !strcmp(x->op, "=");
}

int liveCount(void)
{
    int k, n = 0;
    for (k = 0; k < nq; k++)
        if (!q[k].del) n++;
    return n;
}

void printTAC(void)
{
    int k;
    for (k = 0; k < nq; k++) {
        const Quad *x = &q[k];
        if (x->del) continue;
        if (!strcmp(x->op, "label"))
            printf("%s:\n", x->res);
        else if (!strcmp(x->op, "goto"))
            printf("    goto %s\n", x->res);
        else if (isRel(x->op))
            printf("    if %s %s %s goto %s\n", x->a1, x->op, x->a2, x->res);
        else if (!strcmp(x->op, "print"))
            printf("    print %s\n", x->a1);
        else if (!strcmp(x->op, "="))
            printf("    %s = %s\n", x->res, x->a1);
        else if (!strcmp(x->op, "neg"))
            printf("    %s = -%s\n", x->res, x->a1);
        else

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        printf("    %s = %s %s %s\n", x->res, x->a1, x->op, x->a2);
    }
}

```

optimizer.c

```

/*
 * Code optimizer for the quadruple table
 *
 * 1. Constant propagation and constant folding (data flow analysis, works across loops)
 * 2. Algebraic identities      x+0 x*1 x*0 x-x x^1 x^0 ...
 * 3. Strength reduction      x^2 -> x*x      x*2 -> x+x
 * 4. Merging of temporaries   t = a+b ; x = t -> x = a+b
 * 5. Common sub-expression elimination (inside a basic block)
 * 6. Copy propagation        t2 = t1
 * 7. Dead code elimination    unused temporaries, jumps to the next statement,
 *                             unreferenced labels, unreachable code
 *
 * The passes are repeated until nothing changes, because one pass often creates a
 * chance for another one. Variables of the source program are treated as live at the
 * end of the program, so only temporaries and unreachable code are ever deleted.
 * All arithmetic uses 32 bit int semantics, like C++ on a machine with 32 bit int.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include "compiler.h"

#define MAXV 400                /* variables and temporaries */

enum { S_PROP, S_FOLD, S_ALG, S_STR, S_CSE, S_COPY, S_MERGE, S_DEAD, S_UNREACH, S_COUNT };

static const char *title[S_COUNT] = {
    "Constant propagation", "Constant folding", "Algebraic identities",
    "Strength reduction", "Common sub-expression elimination", "Copy propagation",
    "Merging of temporaries", "Dead code elimination", "Unreachable code elimination"
};
static int count[S_COUNT];

typedef int32_t i32;

/* ----- arithmetic ----- */

static i32 wrap(long long v)
{
    return (i32)(uint32_t)v;
}

static i32 power(i32 base, i32 e)        /* e <= 0 gives 1 */
{
    uint32_t r = 1, x = (uint32_t)base;
    uint32_t n = e > 0 ? (uint32_t)e : 0;
    while (n) {
        if (n & 1) r *= x;
        x *= x;
        n >>= 1;
    }
    return (i32)r;
}

static int evaluate(const char *op, i32 a, i32 b, i32 *r)
{
    switch (op[0]) {
        case '+': *r = wrap((long long)a + b); return 1;
        case '-': *r = wrap((long long)a - b); return 1;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    case '*': *r = wrap((long long)a * b); return 1;
    case '/': if (b == 0 || (a == INT32_MIN && b == -1)) return 0;
               *r = a / b; return 1;
    case '%': if (b == 0 || (a == INT32_MIN && b == -1)) return 0;
               *r = a % b; return 1;
    case '^': *r = power(a, b); return 1;
    }
    return 0;
}

static void setCopy(Quad *x, const char *v)
{
    char tmp[NAMELEN];
    strcpy(tmp, v); /* v may point inside x */
    strcpy(x->op, "=");
    strcpy(x->a1, tmp);
    x->a2[0] = '\0';
}

static int hasA1(const Quad *x)
{
    return defines(x) || isRel(x->op) || !strcmp(x->op, "print");
}

static int hasA2(const Quad *x)
{
    return isBin(x->op) || isRel(x->op);
}

static int nextLive(int i)
{
    for (i++; i < nq; i++)
        if (!q[i].del) return i;
    return -1;
}

static int labelIndex(const char *name)
{
    int k;
    for (k = 0; k < nq; k++)
        if (!q[k].del && !strcmp(q[k].op, "label") && !strcmp(q[k].res, name)) return k;
    return -1;
}

/* ----- 1. constant propagation and folding */

typedef struct { int k; i32 v; } Val; /* k: 1 = constant v, 2 = not a constant */

static char vname[MAXV][NAMELEN];
static int nv = 0;
static Val in[MAXQ][MAXV]; /* value of every variable before quad i */
static char reached[MAXQ];
static int succ[MAXQ][2];

static int varIndex(const char *n)
{
    int k;
    for (k = 0; k < nv; k++)
        if (!strcmp(vname[k], n)) return k;
    if (nv >= MAXV) {
        printf("Error: too many variables\n");
        exit(1);
    }
    strcpy(vname[nv], n);
    return nv++;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

static Val getVal(const Val *st, const char *s)
{
    Val r = { 2, 0 };
    if (isNum(s)) { r.k = 1; r.v = wrap(atoll(s)); }
    else if (isVar(s) || isTemp(s)) r = st[varIndex(s)];
    return r;
}

static void transfer(Val *st, const Quad *x)
{
    Val r = { 2, 0 }, a, b;
    i32 v;
    if (!defines(x)) return;
    a = getVal(st, x->a1);
    if (!strcmp(x->op, "=")) {
        if (a.k == 1) r = a;
    } else if (!strcmp(x->op, "neg")) {
        if (a.k == 1) { r.k = 1; r.v = wrap(-(long long)a.v); }
    } else {
        b = getVal(st, x->a2);
        if (a.k == 1 && b.k == 1 && evaluate(x->op, a.v, b.v, &v)) { r.k = 1; r.v = v; }
    }
    st[varIndex(x->res)] = r;
}

static int meet(Val *dst, const Val *src) /* returns 1 if dst changed */
{
    int v, changed = 0;
    for (v = 0; v < nv; v++) {
        if (dst[v].k == 2) continue;
        if (src[v].k == 2 || dst[v].v != src[v].v) { dst[v].k = 2; changed = 1; }
    }
    return changed;
}

static void substitute(const Val *st, char *name, int *changes)
{
    char num[32];
    Val r;
    if (!isVar(name) && !isTemp(name)) return;
    r = st[varIndex(name)];
    if (r.k != 1) return;
    sprintf(num, "%d", (int)r.v);
    strcpy(name, num);
    count[S_PROP]++;
    (*changes)++;
}

static int constantPropagation(void)
{
    Val out[MAXV];
    int i, v, s, first, changed, changes = 0;

    for (i = 0; i < nq; i++) { /* make a slot for every name */
        if (q[i].del) continue;
        if (defines(&q[i])) varIndex(q[i].res);
        if (hasA1(&q[i]) && (isVar(q[i].a1) || isTemp(q[i].a1))) varIndex(q[i].a1);
        if (hasA2(&q[i]) && (isVar(q[i].a2) || isTemp(q[i].a2))) varIndex(q[i].a2);
    }

    /* control flow graph */
    for (i = 0; i < nq; i++) {
        reached[i] = 0;
        succ[i][0] = succ[i][1] = -1;
        if (q[i].del) continue;
        if (!strcmp(q[i].op, "goto")) {
            succ[i][0] = labelIndex(q[i].res);

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    } else if (isRel(q[i].op)) {
        succ[i][0] = nextLive(i);
        succ[i][1] = labelIndex(q[i].res);
    } else {
        succ[i][0] = nextLive(i);
    }
}

first = nextLive(-1);
if (first < 0) return 0;
reached[first] = 1;
for (v = 0; v < nv; v++) { in[first][v].k = 2; in[first][v].v = 0; }

do {
    changed = 0;
    for (i = 0; i < nq; i++) {
        if (q[i].del || !reached[i]) continue;
        memcpy(out, in[i], sizeof(Val) * nv);
        transfer(out, &q[i]);
        for (s = 0; s < 2; s++) {
            int t = succ[i][s];
            if (t < 0) continue;
            if (!reached[t]) {
                reached[t] = 1;
                memcpy(in[t], out, sizeof(Val) * nv);
                changed = 1;
            } else if (meet(in[t], out)) {
                changed = 1;
            }
        }
    }
} while (changed);

for (i = 0; i < nq; i++) {
    Quad *x = &q[i];
    if (x->del) continue;
    if (!reached[i]) { x->del = 1; count[S_UNREACH]++; changes++; continue; }

    if (hasA1(x)) substitute(in[i], x->a1, &changes);
    if (hasA2(x)) substitute(in[i], x->a2, &changes);

    if (isBin(x->op) && isNum(x->a1) && isNum(x->a2)) {
        i32 r;
        if (evaluate(x->op, wrap(atoll(x->a1)), wrap(atoll(x->a2)), &r)) {
            char num[32];
            sprintf(num, "%d", (int)r);
            setCopy(x, num);
            count[S_FOLD]++; changes++;
        }
    } else if (!strcmp(x->op, "neg") && isNum(x->a1)) {
        char num[32];
        sprintf(num, "%d", (int)wrap(-atoll(x->a1)));
        setCopy(x, num);
        count[S_FOLD]++; changes++;
    } else if (isRel(x->op) && isNum(x->a1) && isNum(x->a2)) {
        i32 a = wrap(atoll(x->a1)), b = wrap(atoll(x->a2));
        int t = !strcmp(x->op, "<") ? a < b : !strcmp(x->op, ">") ? a > b :
            !strcmp(x->op, "<=") ? a <= b : !strcmp(x->op, ">=") ? a >= b :
            !strcmp(x->op, "==") ? a == b : a != b;
        if (t) { strcpy(x->op, "goto"); x->a1[0] = x->a2[0] = '\0'; }
        else x->del = 1;
        count[S_FOLD]++; changes++;
    }
}
return changes;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
/* ----- 2. algebraic identities */
```

```
static int algebraicIdentities(void)
{
    int i, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        char o;
        int done = 1;
        if (x->del || !isBin(x->op)) continue;
        o = x->op[0];
        if (o == '+') {
            if (!strcmp(x->a2, "0")) setCopy(x, x->a1);
            else if (!strcmp(x->a1, "0")) setCopy(x, x->a2);
            else done = 0;
        } else if (o == '-') {
            if (!strcmp(x->a2, "0")) setCopy(x, x->a1);
            else if (!strcmp(x->a1, x->a2)) setCopy(x, "0");
            else done = 0;
        } else if (o == '*') {
            if (!strcmp(x->a1, "0") || !strcmp(x->a2, "0")) setCopy(x, "0");
            else if (!strcmp(x->a2, "1")) setCopy(x, x->a1);
            else if (!strcmp(x->a1, "1")) setCopy(x, x->a2);
            else done = 0;
        } else if (o == '/') {
            if (!strcmp(x->a2, "1")) setCopy(x, x->a1);
            else done = 0;
        } else if (o == '%') {
            if (!strcmp(x->a2, "1")) setCopy(x, "0");
            else done = 0;
        } else if (o == '^') {
            if (!strcmp(x->a2, "1")) setCopy(x, x->a1);
            else if (!strcmp(x->a2, "0")) setCopy(x, "1");
            else done = 0;
        }
        if (done) { count[S_ALG]++; changes++; }
    }
    return changes;
}
```

```
/* ----- 3. strength reduction */
```

```
static int strengthReduction(void)
{
    int i, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        if (x->del || !isBin(x->op)) continue;
        if (x->op[0] == '^' && !strcmp(x->a2, "2") && !isNum(x->a1)) {
            strcpy(x->op, "*"); /* x ^ 2 -> x * x */
            strcpy(x->a2, x->a1);
        } else if (x->op[0] == '*' && !strcmp(x->a2, "2") && !isNum(x->a1)) {
            strcpy(x->op, "+"); /* x * 2 -> x + x */
            strcpy(x->a2, x->a1);
        } else if (x->op[0] == '*' && !strcmp(x->a1, "2") && !isNum(x->a2)) {
            strcpy(x->op, "+"); /* 2 * x -> x + x */
            strcpy(x->a1, x->a2);
        } else continue;
        count[S_STR]++; changes++;
    }
    return changes;
}
```

```
/* ----- uses of a name */
```

```
static int uses(const char *n)
{

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

int i, c = 0;
for (i = 0; i < nq; i++) {
    if (q[i].del) continue;
    if (hasA1(&q[i]) && !strcmp(q[i].a1, n)) c++;
    if (hasA2(&q[i]) && !strcmp(q[i].a2, n)) c++;
}
return c;
}

/* ----- 4. merging of temporaries */

static int mergeTemps(void)
{
    int i, j, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        if (x->del || !defines(x) || !isTemp(x->res)) continue;
        j = nextLive(i);
        if (j < 0) continue;
        if (!strcmp(q[j].op, "=") && !strcmp(q[j].a1, x->res) && uses(x->res) == 1) {
            strcpy(x->res, q[j].res);          /* t = a op b ; x = t */
            q[j].del = 1;
            count[S_MERGE]++; changes++;
        }
    }
    return changes;
}

/* ----- 5. common sub-expression elimination */

typedef struct { char op[8], a1[NAMELEN], a2[NAMELEN], holder[NAMELEN]; } Avail;
static Avail av[MAXQ];
static int  nav;

static void kill(const char *n)          /* n is assigned: forget what depends on it */
{
    int k;
    for (k = 0; k < nav; ) {
        if (!strcmp(av[k].a1, n) || !strcmp(av[k].a2, n) || !strcmp(av[k].holder, n))
            av[k] = av[--nav];
        else
            k++;
    }
}

static int commonSubexpressions(void)
{
    int i, k, changes = 0;
    nav = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        char op[8], a1[NAMELEN], a2[NAMELEN];
        int found = -1;
        if (x->del) continue;
        if (!strcmp(x->op, "label")) { nav = 0; continue; }      /* new basic block */
        if (!defines(x)) continue;
        strcpy(op, x->op); strcpy(a1, x->a1); strcpy(a2, x->a2);

        if (isBin(op)) {
            int comm = (op[0] == '+' || op[0] == '*');
            for (k = 0; k < nav && found < 0; k++) {
                if (strcmp(av[k].op, op)) continue;
                if ((!strcmp(av[k].a1, a1) && !strcmp(av[k].a2, a2)) ||
                    (comm && !strcmp(av[k].a1, a2) && !strcmp(av[k].a2, a1)))
                    found = k;
            }
        }
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        if (found >= 0) {
            char holder[NAMELEN];
            strcpy(holder, av[found].holder);
            setCopy(x, holder);
            count[S_CSE]++; changes++;
        }
        kill(x->res);
        if (found < 0 && isBin(op) && strcmp(a1, x->res) && strcmp(a2, x->res)) {
            strcpy(av[nav].op, op); strcpy(av[nav].a1, a1);
            strcpy(av[nav].a2, a2); strcpy(av[nav].holder, x->res);
            nav++;
        }
    }
    return changes;
}

/* ----- 6. copy propagation ----- */

static int copyPropagation(void)
{
    int i, j, changes = 0;
    for (i = 0; i < nq; i++) {
        Quad *x = &q[i];
        if (x->del || strcmp(x->op, "=") || !isTemp(x->res) || !isTemp(x->a1)) continue;
        for (j = 0; j < nq; j++) {
            /* temporaries are assigned only once */
            if (q[j].del) continue;
            if (hasA1(&q[j]) && !strcmp(q[j].a1, x->res)) strcpy(q[j].a1, x->a1);
            if (hasA2(&q[j]) && !strcmp(q[j].a2, x->res)) strcpy(q[j].a2, x->a1);
        }
        x->del = 1;
        count[S_COPY]++; changes++;
    }
    return changes;
}

/* ----- 7. dead code elimination ----- */

static int labelReferenced(const char *name)
{
    int j;
    for (j = 0; j < nq; j++)
        if (!q[j].del && (!strcmp(q[j].op, "goto") || isRel(q[j].op)) &&
            !strcmp(q[j].res, name)) return 1;
    return 0;
}

static int deadCode(void)
{
    int i, changes = 0, changed = 1;
    while (changed) {
        changed = 0;
        for (i = 0; i < nq; i++) {
            Quad *x = &q[i];
            int n;
            if (x->del) continue;
            n = nextLive(i);
            if (defines(x) && isTemp(x->res) && uses(x->res) == 0)
                x->del = 1; /* result never used */
            else if (!strcmp(x->op, "=") && !strcmp(x->a1, x->res))
                x->del = 1; /* x = x */
            else if ((!strcmp(x->op, "goto") || isRel(x->op)) && n >= 0 &&
                !strcmp(q[n].op, "label") && !strcmp(q[n].res, x->res))
                x->del = 1; /* jump to next quad */
            else if (!strcmp(x->op, "label") && !labelReferenced(x->res))
                x->del = 1; /* nobody jumps here */
            else
                continue;
        }
        if (changes) changed = 1;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        count[S_DEAD]++; changes++; changed = 1;
    }
}
return changes;
}

/* ----- driver */

void optimize(void)
{
    int round, c;
    for (round = 0; round < 20; round++) {
        c = constantPropagation();
        c += algebraicIdentities();
        c += strengthReduction();
        c += mergeTemps();
        c += commonSubexpressions();
        c += copyPropagation();
        c += deadCode();
        if (!c) break;
    }
}

void printOptimizationReport(void)
{
    int k, any = 0;
    printf("Optimizations applied\n");
    for (k = 0; k < S_COUNT; k++)
        if (count[k]) { printf("    %-36s : %d\n", title[k], count[k]); any = 1; }
    if (!any) printf("    none\n");
}

```

codegen.c

```

/*
 * Code generator : optimized quadruples -> 8086 style assembly code
 *
 * Registers : every variable of the source program owns a register (R0, R1, ...).
 *              Temporaries share the registers that follow: a temporary gets a free
 *              register when it is defined and gives it back after its last use, and a
 *              result may reuse the register of an operand that dies in the same quad.
 * Constants : a constant operand is first moved to a scratch register (MOV R, #n).
 * Power      : x ^ n is expanded into repeated MUL (small constant n) or into a loop.
 * Output     : the assembly code is printed and also written to output.asm
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include "compiler.h"

#define MAXR 600

static FILE *out;

static char vname[MAXR][NAMELEN];          /* source variables -> R0 .. R(nvar-1) */
static int  nvar = 0;

typedef struct { char name[NAMELEN]; int reg; int last; } Temp;
static Temp tmp[MAXQ];                      /* temporaries: register and last use */
static int  ntmp = 0;

static char busy[MAXR];                    /* register in use (temporaries only) */
static int  maxReg = -1;
static int  scratch[8], nscratch = 0;
static int  powLabels = 0;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

/* ----- output */

static void ins(const char *fmt, ...)
{
    char buf[200];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(buf, sizeof buf, fmt, ap);
    va_end(ap);
    if (buf[strlen(buf) - 1] == ':') {
        printf("%s\n", buf);
        fprintf(out, "%s\n", buf);
    } else {
        printf("    %s\n", buf);
        fprintf(out, "    %s\n", buf);
    }
}

/* ----- registers */

static int varReg(const char *n)
{
    int k;
    for (k = 0; k < nvar; k++)
        if (!strcmp(vname[k], n)) return k;
    return -1;
}

static void addVar(const char *n)
{
    if (isVar(n) && varReg(n) < 0) strcpy(vname[nvar++], n);
}

static Temp *findTemp(const char *n)
{
    int k;
    for (k = 0; k < ntmp; k++)
        if (!strcmp(tmp[k].name, n)) return &tmp[k];
    strcpy(tmp[ntmp].name, n);
    tmp[ntmp].reg = -1;
    tmp[ntmp].last = -1;
    return &tmp[ntmp++];
}

static int regOf(const char *n)
{
    return isTemp(n) ? findTemp(n)->reg : varReg(n);
}

static int allocReg(void)
{
    int r;
    for (r = nvar; r < MAXR; r++)
        if (!busy[r]) {
            busy[r] = 1;
            if (r > maxReg) maxReg = r;
            return r;
        }
    printf("Error: out of registers\n");
    exit(1);
}

static int newScratch(void)
{
    int r = allocReg();
    scratch[nscratch++] = r;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    return r;
}

static void loadTo(int r, const char *a)
{
    if (isNum(a)) ins("MOV R%d, #s", r, a);
    else if (regOf(a) != r) ins("MOV R%d, R%d", r, regOf(a));
}

static int operandReg(const char *a)          /* register that holds operand a */
{
    int r;
    if (!isNum(a)) return regOf(a);
    r = newScratch();
    ins("MOV R%d, #s", r, a);
    return r;
}

/* Register for the result of quad i. A temporary that dies in this quad lends its
   register to the result. */
static int destReg(int i, const Quad *x)
{
    Temp *t;
    const char *ops[2];
    int k;
    if (!isTemp(x->res)) return varReg(x->res);
    t = findTemp(x->res);
    ops[0] = x->a1; ops[1] = x->a2;
    for (k = 0; k < 2; k++)
        if (isTemp(ops[k]) && findTemp(ops[k])->last == i) {
            t->reg = findTemp(ops[k])->reg;
            return t->reg;
        }
    t->reg = allocReg();
    return t->reg;
}

static void finish(int i, const Quad *x, int d)
{
    const char *ops[2];
    int k;
    ops[0] = x->a1; ops[1] = x->a2;
    for (k = 0; k < 2; k++)
        if (isTemp(ops[k]) && findTemp(ops[k])->last == i && findTemp(ops[k])->reg != d)
            busy[findTemp(ops[k])->reg] = 0;
    if (isTemp(x->res) && findTemp(x->res)->last < 0) busy[d] = 0;    /* never used */
    for (k = 0; k < nscratch; k++) busy[scratch[k]] = 0;
    nscratch = 0;
}

/* ----- instructions */

static const char *mnemonic(char op)
{
    switch (op) {
        case '+': return "ADD";
        case '-': return "SUB";
        case '*': return "MUL";
        case '/': return "DIV";
        default : return "MOD";
    }
}

static const char *jumpFor(const char *rel)
{
    if (!strcmp(rel, "<")) return "JL";
    if (!strcmp(rel, ">")) return "JG";
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!strcmp(rel, "<=")) return "JLE";
    if (!strcmp(rel, ">=")) return "JGE";
    if (!strcmp(rel, "==")) return "JE";
    return "JNE";
}

/* d = a1 ^ a2 */
static void genPower(int d, const Quad *x)
{
    int base = newScratch(), acc = newScratch();
    int e = isNum(x->a2) ? atoi(x->a2) : -1;

    if (isNum(x->a2) && e >= 2 && e <= 16) {                /* repeated multiplication */
        int k;
        loadTo(base, x->a1);
        ins("MOV R%d, R%d", acc, base);
        for (k = 1; k < e; k++) ins("MUL R%d, R%d", acc, base);
    } else if (isNum(x->a2) && e <= 1) {                    /* x ^ 1 or x ^ 0 */
        if (e == 1) loadTo(acc, x->a1);
        else ins("MOV R%d, #1", acc);
    } else {                                              /* loop */
        int cnt = newScratch(), zero = newScratch(), one = newScratch();
        int n = ++powLabels;
        loadTo(base, x->a1);
        loadTo(cnt, x->a2);
        ins("MOV R%d, #1", acc);
        ins("MOV R%d, #0", zero);
        ins("MOV R%d, #1", one);
        ins("P%d:", n);
        ins("CMP R%d, R%d", cnt, zero);
        ins("JLE P%d_end", n);
        ins("MUL R%d, R%d", acc, base);
        ins("SUB R%d, R%d", cnt, one);
        ins("JMP P%d", n);
        ins("P%d_end:", n);
    }
    ins("MOV R%d, R%d", d, acc);
}

static void genBinary(int d, const Quad *x)
{
    const char *m = mnemonic(x->op[0]);
    int r1 = isNum(x->a1) ? -1 : regOf(x->a1);
    int r2 = isNum(x->a2) ? -1 : regOf(x->a2);
    int commutative = (x->op[0] == '+' || x->op[0] == '*');

    if (r2 == d && r1 != d) {                            /* the result register holds operand 2 */
        if (commutative) {
            ins("%s R%d, R%d", m, d, operandReg(x->a1));
        } else {
            int s = newScratch();
            loadTo(s, x->a1);
            ins("%s R%d, R%d", m, s, d);
            ins("MOV R%d, R%d", d, s);
        }
    } else {
        loadTo(d, x->a1);
        ins("%s R%d, R%d", m, d, operandReg(x->a2));
    }
}

void generate(FILE *asmFile)
{
    int i, k;
    out = asmFile;

    /* registers of the variables (in order of first appearance) and last use of temporaries */

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

for (i = 0; i < nq; i++) {
    const Quad *x = &q[i];
    if (x->del) continue;
    if (defines(x)) addVar(x->res);
    if (defines(x) || isRel(x->op) || !strcmp(x->op, "print")) addVar(x->a1);
    if (isBin(x->op) || isRel(x->op)) addVar(x->a2);
    if (isTemp(x->a1)) findTemp(x->a1)->last = i;
    if ((isBin(x->op) || isRel(x->op)) && isTemp(x->a2)) findTemp(x->a2)->last = i;
}

for (i = 0; i < nq; i++) {
    const Quad *x = &q[i];
    int d = -1;
    if (x->del) continue;

    if (!strcmp(x->op, "label")) {
        ins("%s:", x->res);
    } else if (!strcmp(x->op, "goto")) {
        ins("JMP %s", x->res);
    } else if (isRel(x->op)) {
        int r1 = operandReg(x->a1), r2 = operandReg(x->a2);
        ins("CMP R%d, R%d", r1, r2);
        ins("%s %s", jumpFor(x->op), x->res);
    } else if (!strcmp(x->op, "print")) {
        if (isStr(x->a1)) ins("OUT %s", x->a1);
        else if (isNum(x->a1)) ins("OUT #s", x->a1);
        else ins("OUT R%d", regOf(x->a1));
    } else if (!strcmp(x->op, "=")) {
        d = destReg(i, x);
        loadTo(d, x->a1);
    } else if (!strcmp(x->op, "neg")) {
        d = destReg(i, x);
        loadTo(d, x->a1);
        ins("NEG R%d", d);
    } else if (x->op[0] == '^') {
        d = destReg(i, x);
        genPower(d, x);
    } else {
        d = destReg(i, x);
        genBinary(d, x);
    }
    finish(i, x, d);
}

printf("\nRegister Allocation\n");
for (k = 0; k < nvar; k++) printf("    %-10s -> R%d\n", vname[k], k);
if (maxReg >= nvar) printf("    temporaries -> R%d .. R%d\n", nvar, maxReg);
else printf("    temporaries -> none needed\n");
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Input (input.cpp)

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int a = 10;
    int b = 20;
    int c = 2 * 3 + a * 1;
    int d = pow(b, 2);
    int sum = 0, i = 1;
    while (i <= 5) {
        sum = sum + i * 2;
        i = i + 1;
    }
    int x1 = sum + i;
    int x2 = sum + i;
    int e = sum * 2;
    if (a < b) {
        cout << "b is greater" << endl;
    } else {
        cout << "a is greater" << endl;
    }
    cout << c + d << endl;
    return 0;
}
```

Additional input (input2.cpp)

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n = 6, f = 1, k = 2;
    int p = pow(k, n);
    int q = n % 4 + (n - n);
    while (n > 1) {
        f = f * n;
        n = n - 1;
    }
    int r = f / 3 + f * 2;
    int s = f / 3 + f * 2;
    int t = -f + p * 0 + q;
    // a loop with a branch inside
    int j = 0;
    while (j < 4) {
        if (j % 2 == 0)
            cout << j * j << endl;
        else
            cout << f - j << endl;
        j = j + 1;
    }
    cout << r + s << endl;
    cout << t << endl;
    cout << p << endl;
    return 0;
}
```

input2.cpp contains a loop whose values are not known at compile time, an if-else inside the loop, a remainder, a division and a repeated expression.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Commands Used

```
lex scanner.l
yacc -d parser.y
cc lex.yy.c y.tab.c quad.c optimizer.c codegen.c -lfl -o minicompiler
./minicompiler input.cpp
```

Output:

```
● kushaal@Kushaals-MacBook-Air Assignment 9 % flex scanner.l
● kushaal@Kushaals-MacBook-Air Assignment 9 % yacc -d parser.y
● kushaal@Kushaals-MacBook-Air Assignment 9 % gcc lex.yy.c y.tab.c quad.c optimizer.c codegen.c -o minicompiler
● kushaal@Kushaals-MacBook-Air Assignment 9 % ./minicompiler input.cpp
----- PHASE 1: LEXICAL ANALYSIS -----
#include <iostream> - preprocessor directive
#include <cmath> - preprocessor directive
using - keyword
namespace - keyword
std - identifier
; - special character
int - keyword
main - identifier
( - special character
) - special character
{ - special character
int - keyword
a - identifier
= - assignment operator
10 - integer constant
; - special character
int - keyword
b - identifier
= - assignment operator
20 - integer constant
; - special character
int - keyword
c - identifier
= - assignment operator
2 - integer constant
* - arithmetic operator
3 - integer constant
+ - arithmetic operator
a - identifier
* - arithmetic operator
1 - integer constant
; - special character
int - keyword
d - identifier
= - assignment operator
pow - function call
( - special character
b - identifier
, - special character
2 - integer constant
) - special character
; - special character
int - keyword
sum - identifier
= - assignment operator
0 - integer constant
```

Mini compiler output for input.cpp (part 1 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

, - special character
i - identifier
= - assignment operator
1 - integer constant
; - special character
while - keyword
( - special character
i - identifier
<= - relational operator
5 - integer constant
) - special character
{ - special character
sum - identifier
= - assignment operator
sum - identifier
+ - arithmetic operator
i - identifier
* - arithmetic operator
2 - integer constant
; - special character
i - identifier
= - assignment operator
i - identifier
+ - arithmetic operator
1 - integer constant
; - special character
} - special character
int - keyword
x1 - identifier
= - assignment operator
sum - identifier
+ - arithmetic operator
i - identifier
; - special character
int - keyword
x2 - identifier
= - assignment operator
sum - identifier
+ - arithmetic operator
i - identifier
; - special character
int - keyword
e - identifier
= - assignment operator
sum - identifier
* - arithmetic operator
2 - integer constant
; - special character
if - keyword
( - special character

```

Mini compiler output for input.cpp (part 2 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

a - identifier
< - relational operator
b - identifier
) - special character
{ - special character
cout - output stream (function call)
<< - stream insertion operator
"b is greater" - string constant
<< - stream insertion operator
endl - stream manipulator
; - special character
} - special character
else - keyword
{ - special character
cout - output stream (function call)
<< - stream insertion operator
"a is greater" - string constant
<< - stream insertion operator
endl - stream manipulator
; - special character
} - special character
cout - output stream (function call)
<< - stream insertion operator
c - identifier
+ - arithmetic operator
d - identifier
<< - stream insertion operator
endl - stream manipulator
; - special character
return - keyword
0 - integer constant
; - special character
} - special character

```

----- PHASE 2: SYNTAX ANALYSIS -----
 Syntactically correct

```

----- SYMBOL TABLE -----
Name      Type      Bytes      Address      Value
a          int         4           1000          10
b          int         4           1004          20
c          int         4           1008          -
d          int         4           1012          -
sum        int         4           1016          0
i          int         4           1020          1
x1         int         4           1024          -
x2         int         4           1028          -
e          int         4           1032          -

```

Mini compiler output for input.cpp (part 3 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

----- PHASE 3: INTERMEDIATE CODE (TAC) -----

```

a = 10
b = 20
t1 = 2 * 3
t2 = a * 1
t3 = t1 + t2
c = t3
t4 = b ^ 2
d = t4
sum = 0
i = 1
L1:
  if i > 5 goto L2
  t5 = i * 2
  t6 = sum + t5
  sum = t6
  t7 = i + 1
  i = t7
  goto L1
L2:
  t8 = sum + i
  x1 = t8
  t9 = sum + i
  x2 = t9
  t10 = sum * 2
  e = t10
  if a >= b goto L3
  print "b is greater"
  print "\n"
  goto L4
L3:
  print "a is greater"
  print "\n"
L4:
  t11 = c + d
  print t11
  print "\n"

```

Mini compiler output for input.cpp (part 4 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

----- PHASE 4: OPTIMIZED CODE -----

```

a = 10
b = 20
c = 16
d = 400
sum = 0
i = 1

```

```

L1:
  if i > 5 goto L2
  t5 = i + i
  sum = sum + t5
  i = i + 1
  goto L1

```

```

L2:
  x1 = sum + i
  x2 = x1
  e = sum + sum
  print "b is greater"
  print "\n"
  print 416
  print "\n"

```

```

Optimizations applied
  Constant propagation          : 11
  Constant folding              : 6
  Strength reduction            : 2
  Common sub-expression elimination : 1
  Merging of temporaries        : 5
  Dead code elimination         : 8
  Unreachable code elimination  : 2

```

Instructions: 36 -> 20

Mini compiler output for input.cpp (part 5 of 6)

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

----- PHASE 5: TARGET CODE (8086 style) -----

```
MOV R0, #10
MOV R1, #20
MOV R2, #16
MOV R3, #400
MOV R4, #0
MOV R5, #1
```

L1:

```
MOV R9, #5
CMP R5, R9
JG L2
MOV R9, R5
ADD R9, R5
ADD R4, R9
MOV R9, #1
ADD R5, R9
JMP L1
```

L2:

```
MOV R6, R4
ADD R6, R5
MOV R7, R6
MOV R8, R4
ADD R8, R4
OUT "b is greater"
OUT "\n"
OUT #416
OUT "\n"
```

Register Allocation

```
a      -> R0
b      -> R1
c      -> R2
d      -> R3
sum    -> R4
i      -> R5
x1     -> R6
x2     -> R7
e      -> R8
temporaries -> R9 .. R9
```

Assembly code written to output.asm

Mini compiler output for input.cpp (6 of 6)

Phase 1 lists each token with its class. Phase 2 prints Syntactically correct along with the symbol table (type, size, address and initial value). Phase 3 shows the three address code and Phase 4 the optimized code. Phase 5 is the 8086-style assembly code, which is also saved in output.asm.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

In the optimized code, $c = 2*3 + a*1$ is folded to 16 and $d = \text{pow}(b, 2)$ to 400. Since $a < b$ is known to be true, the else part is removed and $c + d$ is printed as the constant 416. Inside the loop $i*2$ became $i+i$, and the second sum $+ i$ is replaced by $x2 = x1$ (common sub-expression). The number of instructions dropped from 36 to 20.

The accepted C++ subset consists of `#include` and `using namespace` lines, `int main()`, `int` declarations, assignments, `if / else`, `while`, `cout << ... << endl` (also `std::cout / std::endl`), the operators `+` `-` `*` `/` `%` and `pow(x, n)` for power, and the relational operators `<` `<=` `>` `>=` `==` `!=`. Declarations are block scoped.

Additional Test Cases:

```

● kushaal@Kushaals-MacBook-Air Assignment 9 % cat input3.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    int b = a + 2
    cout << b << endl;
    return 0;
}
⊗ kushaal@Kushaals-MacBook-Air Assignment 9 % ./minicompiler input3.cpp
----- PHASE 1: LEXICAL ANALYSIS -----
#include <iostream> - preprocessor directive
using - keyword
namespace - keyword
std - identifier
; - special character
int - keyword
main - identifier
( - special character
) - special character
{ - special character
int - keyword
a - identifier
= - assignment operator
5 - integer constant
; - special character
int - keyword
b - identifier
= - assignment operator
a - identifier
+ - arithmetic operator
2 - integer constant
cout - output stream (function call)

----- PHASE 2: SYNTAX ANALYSIS -----
Syntactically incorrect: syntax error near 'cout' at line 7

```

Error handling: syntax, semantic and lexical errors

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Learning Outcomes:

- Understood how the compiler phases (lexical analysis, syntax analysis, intermediate code generation, optimization and code generation) fit together into one working compiler.
- Learnt to use Lex to recognize C++ tokens and pass them to the Yacc parser, and to report lexical errors such as unknown characters and unterminated strings.
- Learnt to write Yacc rules with precedence and associativity for expressions and for declaration, assignment, conditional and looping statements, and to build a symbol table with block scope and semantic checks.
- Learnt to generate three address code with temporaries and labels through syntax directed translation.
- Learnt to apply constant propagation and folding (with data flow analysis so that loops are handled correctly), algebraic identities, strength reduction, common sub-expression elimination and dead code elimination on three address code.
- Learnt to generate 8086-style assembly code with register allocation, comparison and jump instructions, and to build Lex and Yacc programs together with C code.